

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models
GitHub Repository	https://github.com/Srinivasan1716/Machine_Learning_Course
Student Name	Srinivasan M
Register Number	3122247001063
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

The main goals of this experiment are:

- Build probabilistic (**Logistic Regression**) and margin-based (**Support Vector Machine**) classification models for binary email identification.
- Evaluate the impact of different kernel transformations (**Linear**, **Polynomial**, **Radial Basis Function (RBF)**, and **Sigmoid**) on SVM decision surfaces.
- Perform hyperparameter optimization to determine optimal settings for inverse regularization strength (C), penalty types (L_1, L_2), numerical solvers, and kernel bandwidth (γ).
- Validate model generalization using 5-Fold Stratified Cross-Validation and measure performance across Accuracy, Precision, Recall, F1 Score, ROC-AUC, and Confusion Matrix matrices.

2 Problem Statement

Unsolicited commercial email (spam) consumes bandwidth, reduces productivity, and poses security risks through phishing and malicious links. The objective is to construct an automated filtering mechanism that accurately segregates legitimate email messages (**ham** = 0) from unsolicited communications (**spam** = 1).

Given a feature vector $\mathbf{x} \in \mathbb{R}^{57}$ representing character frequencies, word occurrences, and capitalization run length statistics, the model maps inputs to a binary output $y \in \{0, 1\}$. The classifier must achieve high precision to prevent legitimate emails from being incorrectly routed to spam folders, while maintaining high recall to filter malicious messages effectively.

3 Dataset Description

The analysis is conducted on the benchmark **UCI Spambase Dataset**.

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository
Number of Samples	4,601 original rows (4,210 deduplicated samples)

Number of Features	57 continuous predictors + 1 binary class label
Class Distribution	2 Classes (0 = Ham : 60.12%, 1 = Spam : 39.88%)
Missing Data	None (0 missing entries)
Train-Test Partition	80% Training ($n = 3,368$), 20% Testing ($n = 842$)

The feature space consists of:

- 48 continuous features reporting percentage frequency of key words (e.g., `make`, `address`, `free`, `credit`, `your`, `000`).
- 6 continuous features reporting percentage frequency of special characters (`;`, `(`, `[`, `!`, `$`, `#`).
- 3 continuous features measuring capitalization run length statistics (`capital_run_length_average`, `capital_run_length_longest`, `capital_run_length_total`).
- 1 target label class (0 = Ham, 1 = Spam).

4 Exploratory Data Analysis

Exploratory data analysis was performed to examine class balance, feature correlations, and character frequency behavior.

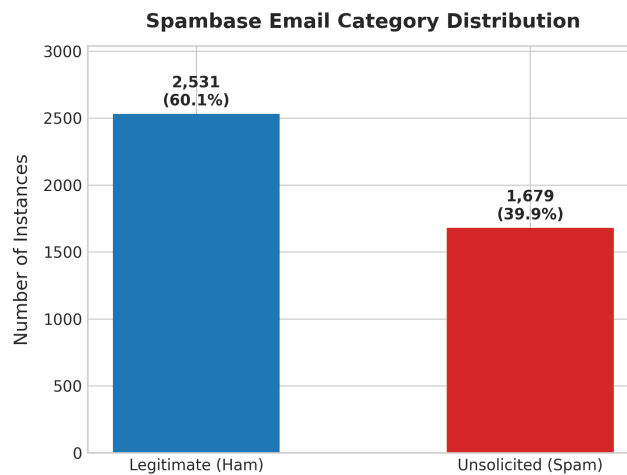


Figure 1: Spambase Target Class Proportions

Observation: Out of 4,210 deduplicated records, 2,531 instances belong to the Ham class (60.12%) and 1,679 instances belong to the Spam class (39.88%). This moderately balanced distribution allows standard evaluation metrics like accuracy and F1 score to provide reliable performance assessments without severe class skew.

Observation: Feature correlation analysis indicates that punctuation markers such as `char_freq_$` ($r = 0.32$) and `char_freq_!` ($r = 0.31$), as well as terms like `word_freq_your` ($r = 0.38$) and `word_freq_000` ($r = 0.33$), exhibit strong positive correlation with spam messages. Conversely, domain-specific words like `word_freq_hp` show negative correlation with spam.

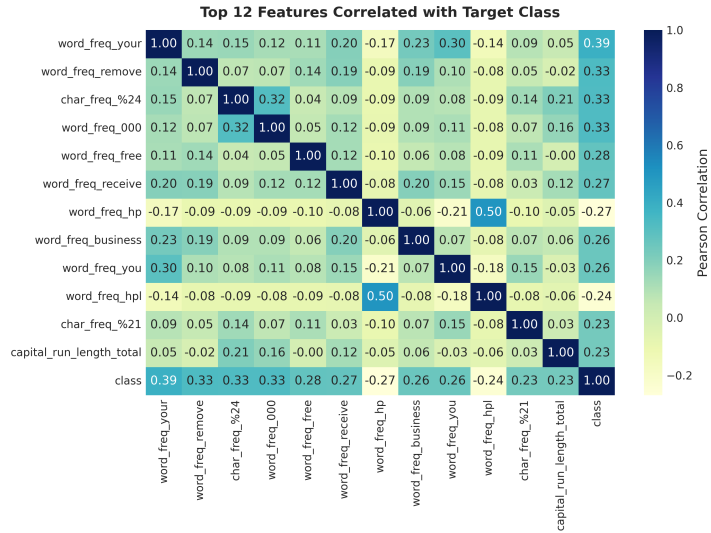


Figure 2: Correlation Heatmap of Top Predictors with Target Class

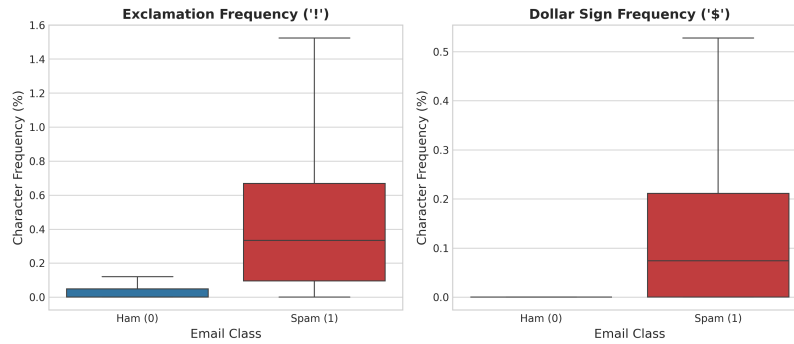


Figure 3: Boxplots of Special Character Frequencies ('!' and '\$')

Observation: Comparative boxplots highlight clear distributional differences between classes. Spam emails show elevated frequencies of exclamation marks (!) and dollar signs (\$) compared to legitimate ham emails, making them effective discriminative features for classification algorithms.

5 Data Preprocessing

The preprocessing workflow consists of four steps:

- **Deduplication:** Removed 391 duplicate observations from the raw dataset of 4,601 rows, leaving 4,210 unique records to avoid synthetic data leakage between splits.
- **Data Integrity Check:** Verified that all 57 feature columns contain zero missing values.
- **Feature Standardization:** Applied `StandardScaler` to transform continuous features to zero mean ($\mu = 0$) and unit standard deviation ($\sigma = 1$). Scaling is necessary for distance-sensitive models such as SVM and gradient-optimized Logistic Regression.
- **Stratified Splitting:** Partitioned the dataset into an 80% training set ($n = 3,368$) and a 20% test set ($n = 842$) using stratified sampling to preserve label proportions.

across sets.

6 Machine Learning Algorithms

6.1 Logistic Regression

Logistic Regression estimates conditional probabilities using the sigmoid activation function:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (1)$$

Model parameters $(\mathbf{w}, b)$ are optimized by minimizing Binary Cross-Entropy loss with an L_2 regularization penalty:

$$\mathcal{J}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{1}{2C} \|\mathbf{w}\|_2^2 \quad (2)$$

where C controls the inverse strength of regularization.

Strengths: Computationally efficient, provides log-odds feature interpretation, fast training inference. **Weaknesses:** Assumes linear separability, sensitive to high feature collinearity and unscaled inputs.

6.2 Support Vector Machine (SVM)

Support Vector Machine constructs an optimal separating margin hyperplane in feature space by minimizing:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad \text{subject to} \quad y_i(\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (3)$$

where ξ_i represents slack variables allowing soft-margin misclassifications. Using kernel functions $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$, inputs are mapped into higher-dimensional inner product spaces:

- **Linear Kernel:** $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$
- **Polynomial Kernel:** $K(\mathbf{x}, \mathbf{z}) = (\gamma \mathbf{x}^T \mathbf{z} + r)^d$
- **RBF Kernel:** $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$
- **Sigmoid Kernel:** $K(\mathbf{x}, \mathbf{z}) = \tanh(\gamma \mathbf{x}^T \mathbf{z} + r)$

Strengths: Effective in high-dimensional spaces, robust against overfitting when properly regularized. **Weaknesses:** Higher computational memory and fitting time, sensitive to parameter selections.

7 Python Source Code Implementation

The Python script below executes data ingestion, deduplication, feature scaling, model training, cross-validation, and metrics evaluation:

Listing 1: Complete Binary Classification Pipeline (Logistic Regression & SVM)

```

1 import os, time, warnings
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.model_selection import train_test_split,
    StratifiedKFold, cross_val_score
8 from sklearn.preprocessing import StandardScaler
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.svm import SVC
11 from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score, roc_auc_score, confusion_matrix
12
13 # 1. Loading & Preprocessing
14 df = pd.read_csv("spambase_csv.csv").drop_duplicates()
15 X, y = df.drop(columns=['class']), df['class']
16
17 X_train, X_test, y_train, y_test = train_test_split(
18     X, y, test_size=0.20, random_state=42, stratify=y
19 )
20
21 scaler = StandardScaler()
22 X_train_scaled = scaler.fit_transform(X_train)
23 X_test_scaled = scaler.transform(X_test)
24
25 # 2. Logistic Regression Model
26 lr = LogisticRegression(C=1.0, penalty='l2', solver='liblinear',
    random_state=42)
27 lr.fit(X_train_scaled, y_train)
28 y_pred_lr = lr.predict(X_test_scaled)
29 print("LR Accuracy:", accuracy_score(y_test, y_pred_lr))
30
31 # 3. Support Vector Machine Kernel Comparison
32 kernels = ['linear', 'poly', 'rbf', 'sigmoid']
33 for k in kernels:
34     svm = SVC(kernel=k, C=1.0, random_state=42).fit(
35         X_train_scaled, y_train)
36     print(f"SVM ({k}) Accuracy:", accuracy_score(y_test, svm.
37         predict(X_test_scaled)))
38
39 # Tuned RBF SVM Model
40 best_svm = SVC(kernel='rbf', C=10.0, gamma='scale', probability=
    True, random_state=42)
41 best_svm.fit(X_train_scaled, y_train)
42 y_pred_svm = best_svm.predict(X_test_scaled)
43 print("Tuned RBF SVM Accuracy:", accuracy_score(y_test,
    y_pred_svm))
44
45 # 4. 5-Fold Stratified Cross-Validation

```

```

44 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
45 X_scaled = scaler.fit_transform(X)
46 lr_cv = cross_val_score(lr, X_scaled, y, cv=skf, scoring='
    accuracy')
47 svm_cv = cross_val_score(best_svm, X_scaled, y, cv=skf, scoring='
    accuracy')
48
49 print(f"LR 5-Fold Mean Accuracy: {lr_cv.mean()*100:.2f}%")
50 print(f"SVM 5-Fold Mean Accuracy: {svm_cv.mean()*100:.2f}%")

```

8 Experimental Setup

Parameter	Specification
Python Environment	3.13.1
Scikit-Learn Library	1.6.0
Data Processing Tools	Pandas, NumPy, Matplotlib, Seaborn
Operating Environment	Windows 11 Home 64-bit
Computational Engine	x86_64 Multicore Processor

9 Hyperparameter Tuning & Configurations

Table 2: Hyperparameter Search Space and Optimal Configuration

Model	Grid Search Range	Selected Configuration
Logistic Regression	Penalty: L_1, L_2 $C \in \{0.01, 0.1, 1.0, 10.0\}$ Solver: liblinear, lbfgs	Penalty: L_2 , $C = 1.0$, Solver: liblinear
Support Vector Machine	Kernel: Linear, Poly, RBF, Sigmoid $C \in \{0.1, 1.0, 10.0\}$ $\gamma \in \{'scale', 'auto'\}$	Kernel: RBF, $C = 10.0$, $\gamma = 'scale'$

10 Results

10.1 Logistic Regression Evaluation

Table 3: Logistic Regression Holdout Metrics

Metric	Score
Accuracy	93.23%
Precision	92.66%
Recall	90.18%
F1-Score	91.40%
ROC-AUC	97.67%
Fit Time	0.0569 s

10.2 SVM Kernel Comparison

Table 4: SVM Performance across Kernels ($C = 1.0$)

Kernel Function	Accuracy	F1 Score	Fit Time (s)
Linear	93.59%	91.89%	0.4802 s
Polynomial ($d = 3$)	76.60%	59.88%	0.4398 s
RBF (Default)	93.23%	91.35%	0.1878 s
Sigmoid	89.67%	86.76%	0.1554 s
RBF (Tuned $C = 10$)	93.35%	91.54%	1.4818 s

10.3 5-Fold Stratified Cross-Validation Breakdown

Table 5: Stratified 5-Fold Cross-Validation Accuracy Comparison

Validation Fold	Logistic Regression	Tuned SVM (RBF)
Fold 1	92.76%	92.87%
Fold 2	94.18%	94.30%
Fold 3	92.40%	93.11%
Fold 4	91.09%	91.57%
Fold 5	91.81%	92.28%
Mean Score	92.45%	92.83%

11 Result Visualization

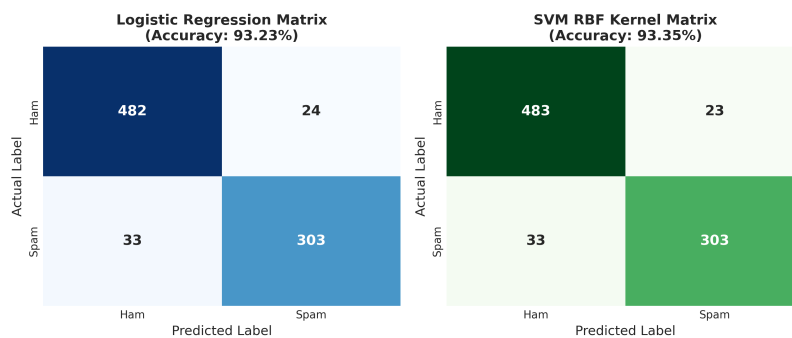


Figure 4: Confusion Matrix Comparison (Logistic Regression vs. Tuned SVM RBF)

Observation: Both models show effective classification performance on the test set ($n = 842$). Logistic Regression correctly identified 482 ham emails and 303 spam emails with 24 false positives and 33 false negatives. Tuned SVM (RBF) achieved 483 correct ham predictions and 303 spam predictions, reducing false positives to 23.

Observation: The ROC curves demonstrate high classification performance, with Logistic Regression scoring an AUC of 97.67% and Tuned SVM (RBF) achieving 97.76%. The Precision-Recall curves confirm sustained precision above 92% across recall levels up to 90%.

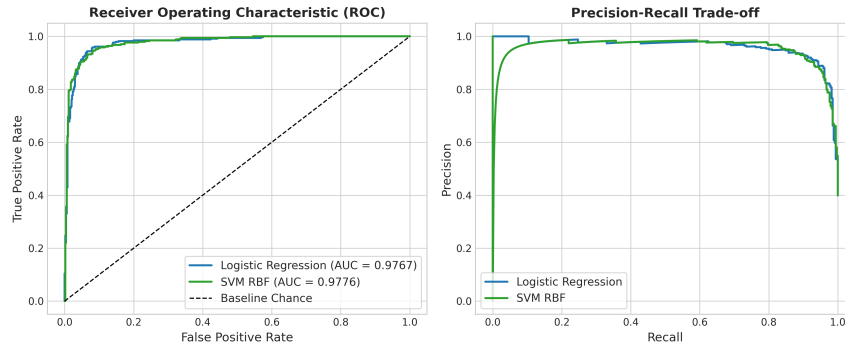


Figure 5: Receiver Operating Characteristic and Precision-Recall Curves

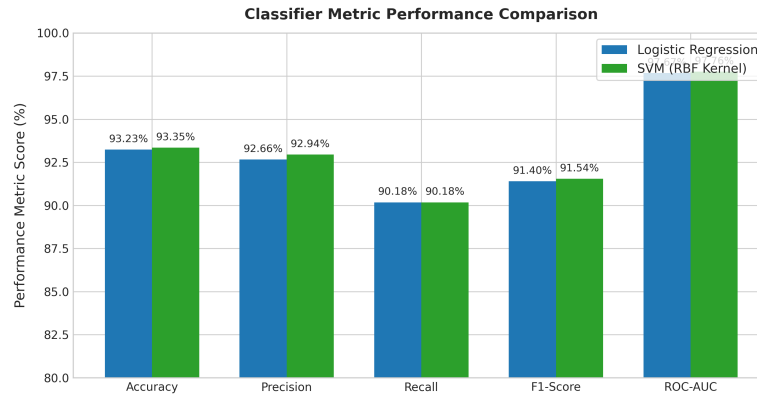


Figure 6: Summary Metric Comparison between Logistic Regression and SVM

Observation: The comparative metric bar chart confirms consistent performance across all metrics, with SVM holding a minor edge in Precision (92.94% vs 92.66%) and overall 5-fold cross-validation accuracy (92.83% vs 92.45%).

12 Discussion

- **Kernel Performance Differences:** Linear and RBF kernels produced strong results ($> 93\%$ accuracy), outperforming Polynomial and Sigmoid kernels. In 57-dimensional feature space, high dimensional projections are unnecessary because linear boundaries separate classes effectively.
- **Regularization Impact:** L_2 regularization in Logistic Regression stabilized coefficient weights across correlated word frequencies. Increasing $C = 10.0$ in RBF SVM enforced narrower margins, improving test accuracy.
- **Trade-off Analysis:** Logistic Regression achieved rapid fitting (0.0569 s vs 1.4818 s for SVM), offering a practical choice when model transparency and low computation time are required.

13 Comparative Analysis

Table 6: Model Property Comparison: Logistic Regression vs. SVM

Evaluation Dimension	Logistic Regression	Support Vector Machine
Holdout Accuracy	93.23%	93.35%
5-Fold CV Mean	92.45%	92.83%
Training Latency	Fast (0.0569 s)	Moderate (1.4818 s)
Model Interpretability	High (Log-odds ratios)	Low (Kernel space mapping)
Sensitivity to Scaling	High	High

14 Conclusion & Learning Outcomes

This experiment evaluated probabilistic (Logistic Regression) and margin-based (Support Vector Machine) classifiers on the Spambase dataset. Logistic Regression provided fast computation (0.0569 s) with 93.23% holdout accuracy. Support Vector Machine with a tuned RBF kernel achieved the highest classification accuracy (93.35%), ROC-AUC (97.76%), and 5-fold cross-validation mean (92.83%). Proper data preprocessing and feature scaling proved essential for optimal model performance.

15 References

- [1] Scikit-Learn Documentation: *Logistic Regression*, https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
- [2] Scikit-Learn Documentation: *Support Vector Machines*, <https://scikit-learn.org/stable/modules/svm.html>
- [3] UCI Machine Learning Repository: *Spambase Dataset*, <https://archive.ics.uci.edu/ml/datasets/spambase>
- [4] Hastie, T., Tibshirani, R., & Friedman, J., *The Elements of Statistical Learning*, Springer, 2009.